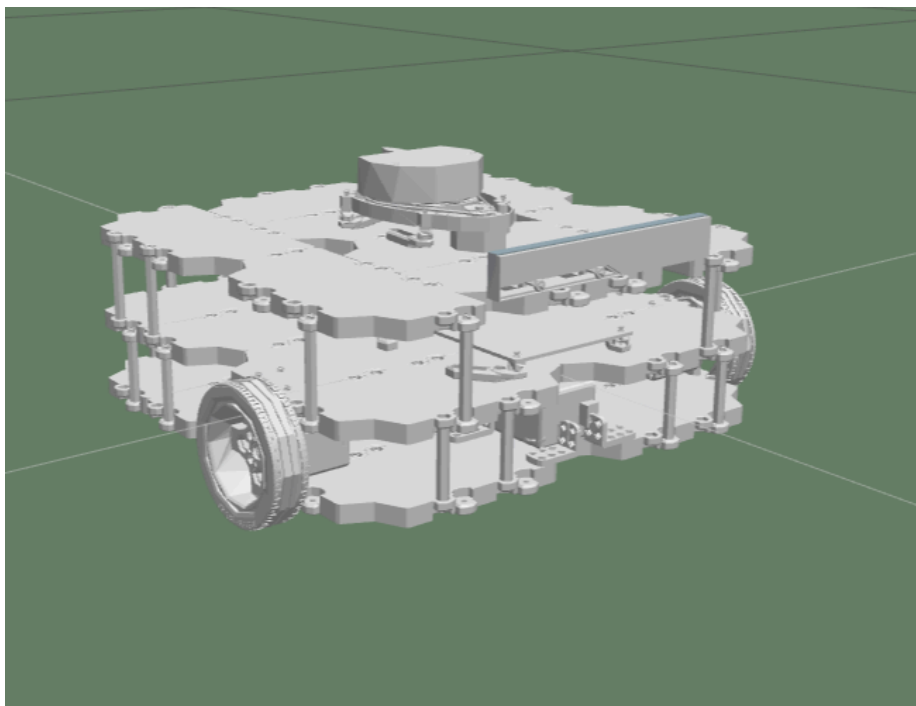


RA: Robòtica Mòbil 2026

Pràctiques

1- Recordant ROS



Dr. Anaís Garrell Zulueta

Departament d'Enginyeria de Sistemes, Automàtica i Informàtica Industrial

CONTINGUTS

- [1. Què Aprendrem?](#)
- [2. Movem el turtlebot 3 burger amb ROS](#)
- [3. Què és un paquet? \(package\)](#)
- [4. Què és un arxiu launch?](#)
- [5. Creació d'un paquet.](#)
- [6. Compilació del paquet](#)
 - [6.1 catkin make](#)
- [7. Primer programa de ROS](#)
- [8. Modificar el fitxer CMakeLists.txt](#)
- [9. Nodes de ROS](#)
- [10. Parameter Server](#)
- [11. ROSCORE](#)
- [12.SIMULACIÓ EN ROS: GAZEBO](#)
 - [12.1 Instal·lació del paquet de simulació](#)
 - [2.2 Launch del món de simulació](#)
 - [2.2.1 Empty World](#)
 - [2.2.2 TurtleBot3 World](#)
 - [2.2.3 TurtleBot3 House](#)
 - [12.3 Operar el turtlebot3](#)
 - [12.4 Visualització en RVIZ](#)
- [13. Simulació de SLAM](#)
 - [13.1 Launch demo món en simulació](#)
 - [13.2 Executar el node de SLAM](#)
 - [13.3 Executar el node de teleoperació](#)
 - [13.4 Guardem el mapa](#)
- [14. Recopilatori](#)

1. Què Aprendre?

- Com estructurar i llançar programes ROS (paquets i fitxers launch)
- Com crear programes ROS bàsics (basats en C++)
- Conceptes bàsics de ROS: Nodes, Servidor de paràmetres, Variables d'entorn, Roscore
- Llançarem el robot virtual anomenat TurtleBot3.
- Utilitzarem l'entorn de simulació Gazebo
- Utilitzarem node de SLAM
- Dibuixar i guardar un mapa de l'entorn de treball

2. Movem el turtlebot 3 burger amb ROS

Per tal de moure el vostre robot, la manera més senzilla és executar un programa de ROS que ja existeix. Un programa ROS s'executa utilitzant fitxers especials anomenats **launch files**.

Per sort, ja existeix un programa ROS fet anteriorment que us permet moure el robot mitjançant el teclat, iniciem aquest programa ROS per teleoperar el robot.

Al PC remot, obriu una terminal: (T1 vol dir terminal 1, T2 terminal 2,...)

T1: `$ roscore`

Al PC remot (el vostre ordinador), obriu una altra terminal. Per tal de connectar-vos al robot, recordeu que el password és **turtlebot**.

Nota: A atenea teniu un document amb les IP's dels robots.

T2: `$ ssh ubuntu@{IP_ADDRESS_OF_RASPBERRY_PI}`
`$ roslaunch turtlebot3_bringup turtlebot3_robot.launch`

Obriu una altra terminal per activar la teleoperació:

T3: `$ roslaunch turtlebot3_teleop turtlebot3_teleop_key.launch`

Ara, podeu utilitzar les tecles indicades a la sortida de WebShell per moure el robot. Les claus bàsiques són les següents:

```
Control Your Turtlebot3
Moving around
    w
a    s    d
    x
w/x : increase/decrease linear velocity
a/d : increase/decrease angular velocity
space key, s : force stop
CTRL-C to quit
```

Quan acabeu, premeu Ctrl+C per aturar l'execució del programa a totes les terminals.

roslaunch és l'ordre que s'utilitza per llançar un programa ROS. La seva estructura és la següent:

```
roslaunch <package_name> <launch_file>
```

Podeu comprovar que la ordre roslaunch té dos paràmetres: el primer és el **nom del paquet que conté el fitxer launch**, i el segon és el **nom del propi fitxer launch** (que s'emmagatzema dins del paquet).

3. Què és un paquet? (package)

És important saber que ROS utilitza els **paquets (packages)** per organitzar els seus programes.

L'estructura bàsica d'un paquet es presenta a continuació:

- Carpeta **launch** : conté fitxers launch (són els executables)
- Carpeta **src**: fitxers de codi (cpp, python)
- **CMakeLists.txt**: Llista de regles de cmake per a la compilació
- **package.xml**: informació del paquet i dependències

Per anar a qualsevol paquet ROS que tingueu en el vostre ordinador, ROS us ofereix una ordre anomenada **roscd**. En escriure:

```
roscd <package_name>
```

Us portarà al camí on es troba el paquet <package_name>.

exemple: Aneu a la terminal:

```
$ roscd turtlebot3_teleop
$ ls
```

Us apareixerà el següent:

```

agarrell@msirilab:~$ roscd turtlebot3_teleop/
agarrell@msirilab:~/catkin_ws/src/turtlebot3/turtlebot3_teleop$ ls
CHANGELOG.rst  CMakeLists.txt  launch  nodes  package.xml  setup.py  src
agarrell@msirilab:~/catkin_ws/src/turtlebot3/turtlebot3_teleop$ 

```

- Cada programa ROS que voleu executar està organitzat en un paquet.
- Cada programa ROS que creeu s'haurà d'organitzar en un paquet.
- Els paquets són el principal sistema d'organització dels programes ROS.

4. Què és un arxiu launch?

Per tal de comprendre què és un arxiu **launch** obriu la carpeta launch dins del paquet **turtlebot3_teleop** i comproveu el fitxer **turtlebot3_teleop_key.launch**:

```

$ roscd turtlebot3_teleop
$ cd launch
$ cat turtlebot3_teleop_key.launch

```

```

<launch>
  <arg name="model" default="$(env TURTLEBOT3_MODEL)" doc="model type [burger,
waffle, waffle_pi]"/>
  <param name="model" value="$(arg model)"/>

  <!-- turtlebot3_teleop_key already has its own built in velocity smoother -->
  <node pkg="turtlebot3_teleop" type="turtlebot3_teleop_key"
name="turtlebot3_teleop_keyboard" output="screen">
  </node>
</launch>

```

Al fitxer launch, teniu algunes etiquetes addicionals per configurar paràmetres i reassignacions. De moment, centreu-vos en l'etiqueta del node. Us heu de fixar en el què està en negreta.

Tots els fitxers launch estan continguts dins d'una etiqueta <launch>. Dins d'aquesta etiqueta, podeu veure una etiqueta <node>, on especifiquem els paràmetres següents:

1. **pkg="package_name"** # Nom del paquet que conté el codi del programa ROS a executar
2. **type="cpp_executable_name"** # Nom del fitxer executable cpp que volem executar
3. **name="node_name"** # Nom del node ROS que llançarà el nostre fitxer C++
4. **output="type_of_output"** # A través de quin canal imprimireu la sortida del programa.

5. Creació d'un paquet.

Fins ara hem estat comprovant l'estructura d'un paquet ja construït. Quan volem crear paquets, hem de treballar en un espai de treball ROS molt específic, que es coneix com **catkin_workspace**. L'espai de treball catkin és el directori del vostre disc dur on han de residir els vostres propis paquets ROS per tal que ROS els pugui utilitzar.

Normalment, el directori de l'espai de treball catkin s'anomena **catkin_ws**.

```
$ roscd
$ cd..
$ pwd
```

```
user~$ pwd
/home/user/catkin_ws
```

Dins del workspace, hi ha un directori anomenat **src**. Aquesta carpeta contindrà tots els paquets creats. Cada vegada que vulgueu crear un paquet, heu d'estar en aquest directori (catkin_ws/src). Escriuiu al vostre terminal **cd src** per moure's al directori **source**.

```
$ cd src
```

Per crear un paquet, escriviu a la vostre terminal:

```
$ catkin_create_pkg my_package roscpp
```

Això crearà dins del nostre directori "src" un paquet nou amb alguns fitxers.

Substituïu el nom **my_package** per **my_package_el_vostre_nom**.

Abans de crear un nou paquet, anem a veure com es construeix aquesta comanda:

```
catkin_create_pkg <package_name> <package_dependencies>
```

El **package_name** és el nom del paquet que voleu crear, i **package_dependencies** són els noms d'altres paquets ROS dels quals depèn el vostre paquet.

Per comprovar que el nostre paquet s'ha creat correctament, podem utilitzar algunes ordres ROS relacionades amb els paquets. Per exemple, escrivim:

```
$ rospack list
$ rospack list | grep my_package
$ roscd my_package
```

rospack list: us ofereix una llista amb tots els paquets del vostre sistema ROS.

rospack list | grep my_package: filtra, de tots els paquets que es troben al sistema ROS, el paquet anomenat *my_package*.

roscd my_package: us porta a la ubicació anomenada *my_package*.

6. Compilació del paquet

Quan creeu un paquet, normalment haureu de compilar-lo per tal que funcioni. Hi ha diferents mètodes que es poden utilitzar per compilar els vostres paquets ROS. Per a aquest curs us presentarem el més habitual, **catkin_make**:

6.1 catkin make

Una de les opcions més populars utilitzades pels desenvolupadors de ROS per compilar els seus paquets és la següent:

```
catkin_make
```

Aquesta ordre compilarà tot el vostre directori **src** i s'ha de fer **al vostre directori catkin_ws per funcionar**. Això és **OBLIGATORI**. Si intenteu compilar des d'un altre directori, no funcionarà.

Aneu al vostre directori **catkin_ws** i compileu la vostra carpeta font.

```
$ roscd  
$ cd..  
$ catkin_make
```

Després de compilar, també és molt important actualitzar el vostre espai de treball.

D'aquesta manera s'assegurarà que ROS sempre rebrà els darrers canvis fets al vostre espai de treball.

```
$ source devel/setup.bash
```

De vegades (per exemple, en projectes grans) no voldreu compilar tots els vostres paquets, sinó només aquell(s) on heu fet canvis. Podeu fer-ho amb l'ordre següent:

```
catkin_make --only-pkg-with-deps <package_name>
```

Aquesta ordre només compilarà els paquets especificats i les seves dependències. Intenteu compilar el vostre paquet anomenat **my_package** amb aquesta ordre.

```
$catkin_make --only-pkg-with-deps my_package
```

7. Primer programa de ROS

Ara ja teniu el paquet creat, anem a realitzar un programa en ROS.

1. Creeu al directori **src** del **my_package_vostre_nom** un fitxer C++ que s'executarà. Per a aquest exercici, només cal que copieu aquest codi C++ **first_program.cpp**.

```
$ touch first_program.cpp
```

Obriu l'arxiu amb l'editor que preferiu, vim, gedit...

```
#include <ros/ros.h>

int main(int argc, char** argv) {

    ros::init(argc, argv, "Remembering_ROS");
    ros::NodeHandle nh;
    ROS_INFO("Trying to remember ROS....");
    ros::spinOnce();
    return 0;
}
```

2. Creeu un directori de launch dins del paquet anomenat my_package:

```
$ roscd my_package
$ mkdir launch
```

3. Creeu un nou fitxer .launch dins del directori launch:

```
$ touch launch/my_package_launch_file.launch
```

4. Omple aquest fitxer launch de la manera següent: (recordeu que el paquet es diu **my_package_el_vostre_nom** i s'ha de substituir per **my_package**)

```
<launch>
  <!-- My Package launch file -->
  <node pkg="my_package" type="first_program" name="Remembering_ROS"
output="screen">
  </node>
</launch>
```

5. Modifiqueu el fitxer **CMakeLists.txt** per tal de generar un executable a partir del fitxer C++ que acabeu de crear.

A la secció **Build** del fitxer **CMakeLists.txt**, afegiu les línies següents:


```

add_executable(first_program src/first_program.cpp)
add_dependencies(first_program      ${first_program_EXPORTED_TARGETS}
${catkin_EXPORTED_TARGETS})
target_link_libraries(first_program
    ${catkin_LIBRARIES}
)

```

```

139 ## e.g. "roslaunch someones_pkg node" instead of "roslaunch someones_pkg someones_pkg_node"
140 # set_target_properties(${PROJECT_NAME}_node PROPERTIES OUTPUT_NAME node PREFIX "")
141
142 ## Add cmake target dependencies of the executable
143 ## same as for the library above
144 # add_dependencies(${PROJECT_NAME}_node ${${PROJECT_NAME}_EXPORTED_TARGETS} ${catkin_EXPORTED_TARGETS})
145
146 ## Specify libraries to link a library or executable target against
147 # target_link_libraries(${PROJECT_NAME}_node
148 #     ${catkin_LIBRARIES}
149 # )
150
151 add_executable(simple src/simple.cpp)
152 add_dependencies(simple ${simple_EXPORTED_TARGETS} ${catkin_EXPORTED_TARGETS})
153 target_link_libraries(simple
154     ${catkin_LIBRARIES}
155 )
156
157 #####
158 ## Install ##
159 #####
160
161 # all install targets should use catkin DESTINATION variables
162 # See http://ros.org/doc/api/catkin/html/adv_user_guide/variables.html
163

```

6. Compila el paquet tal com s'ha explicat anteriorment.

```

$ roscd; cd..
$ catkin_make

```

Si tot va bé, hauríeu d'obtenir una sortida així:

```

[ 0%] Built target gazebo_msgs_generate_messages_nodejs
[ 0%] Built target std_srvs_generate_messages_cpp
[ 0%] Built target trajectory_msgs_generate_messages_nodejs
[ 0%] Built target gazebo_msgs_generate_messages_py
Scanning dependencies of target first_program
[ 0%] Built target trajectory_msgs_generate_messages_py
[ 0%] Built target dynamic_reconfigure_generate_messages_cpp
[ 0%] Built target turtlebot3_description_xacro_generated_to_devel_space_
[ 6%] Built target turtlebot3_msgs_generate_messages_py
[ 10%] Built target turtlebot3_msgs_generate_messages_cpp
[ 15%] Built target turtlebot3_msgs_generate_messages_nodejs
[ 21%] Built target turtlebot3_msgs_generate_messages_eus
[ 26%] Built target turtlebot3_msgs_generate_messages_lisp
[ 39%] Built target turtlebot3_example_generate_messages_py
[ 42%] Built target flat_world_imu_node
[ 53%] Built target turtlebot3_example_generate_messages_nodejs
[ 64%] Built target turtlebot3_example_generate_messages_lisp
[ 76%] Built target turtlebot3_example_generate_messages_eus
[ 87%] Built target turtlebot3_example_generate_messages_cpp
[ 89%] Building CXX object my_package/CMakeFiles/first_program.dir/src/first_program.cpp.o
[ 89%] Built target turtlebot3_msgs_generate_messages
[ 90%] Linking CXX executable /home/agarrell/catkin_ws/devel/lib/turtlebot3_gazebo/turtlebot3_drive
[ 93%] Built target turtlebot3_fake_node
[ 96%] Built target turtlebot3_diagnostics
[ 96%] Built target turtlebot3_example_generate_messages
[ 98%] Built target turtlebot3_drive
[100%] Linking CXX executable /home/agarrell/catkin_ws/devel/lib/my_package/first_program
[100%] Built target first_program
agarrell@msirlab:~/catkin_ws$

```

7. Finalment, executeu l'ordre roslaunch a la terminal per iniciar el vostre programa:

```
T1: $ roscore
T2: $ roslaunch my_package my_package_launch_file.launch
```

Hauríeu de veure la cita de “*Trying to remember ROS....*” entre la sortida de l'ordre roslaunch.

De vegades, ROS no detectarà un paquet nou quan l'acabeu de crear, de manera que no podreu fer un roslaunch. En aquest cas, podeu forçar ROS a actualitzar la llista de paquets amb l'ordre:

```
T1: $ rospack profile
```

Ara anem a recordar l'explicació del codi línia per línia:

```
#include <ros/ros.h>
// Here we are including all the headers necessary to use the most common
public pieces of the ROS system.
// Always we create a new C++ file, we will need to add this include.

int main(int argc, char** argv) { // We start the main C++ program

    ros::init(argc, argv, "Remembering_ROS"); // We initiate a ROS node
called Remembering_ROS

    ros::NodeHandle nh; // We create a handler for the node. This handler
will actually do the initialization of the node

    ROS_INFO("Trying to remember ROS...."); // This is the same as a print in
ROS

    ros::spinOnce(); // Calling ros::spinOnce() here is not necessary for
this first program, because we are not receiving any callbacks.
However, if you were to add a subscription into this application, and did
not have ros::spinOnce() here, your callbacks would never get called. So,
add it for good measure.

    return 0; // We end our program
}
```

NOTA: Si creeu el vostre fitxer C++ des de la terminal, pot passar que es creï sense permisos d'execució. Si això passa, ROS no el podrà trobar. Si aquest és el vostre cas, podeu donar permisos d'execució al fitxer escrivint la següent comanda: ****chmod +x name_of_the_file.cpp****

Hauríeu de tenir alguna cosa semblant a això al vostre my_package_launch_file.launch:

Nota: tingueu en compte que a l'exemple següent, el nom de l'executable C++ al tipus d'atribut s'anomena `first_program`. Per tant, si heu nomenat el vostre executable C++ amb un nom diferent, això serà diferent.

```
<launch>
  <!-- My Package launch file -->
  <node pkg="my_package" type="first_program" name="Remembering_ROS"
output="screen">
  </node>
</launch>
```

8. Modificar el fitxer CMakeLists.txt

Quan programeu amb C++, serà necessari crear executables dels vostres programes per poder executar-los. Per això, haureu de modificar el fitxer CMakeLists.txt del vostre paquet, per indicar que voleu crear un executable del vostre fitxer C++.

Per fer-ho, heu d'afegir algunes línies al fitxer CMakeLists.txt. De fet, aquestes línies ja estan a l'arxiu, però estan comentades. També podeu trobar-los i deixar-los de comentar. El que vulguis.

A l'exercici anterior, teníeu les línies següents:

```
add_executable(first_program src/first_program.cpp)
add_dependencies(first_program      ${first_program_EXPORTED_TARGETS}
${catkin_EXPORTED_TARGETS})
target_link_libraries(first_program
  ${catkin_LIBRARIES}
)
```

Però... què fan exactament aquestes línies de codi? Bé, bàsicament fan el següent:

```
add_executable(first_program src/first_program.cpp)
```

Aquesta línia **genera un executable a partir del fitxer `first_program.cpp`**, que es troba a la carpeta `src` del vostre paquet. Aquest executable es **col·locarà de manera predeterminada al directori de paquets del vostre espai de treball**, que es troba per defecte a `~/catkin_ws/devel/lib/`.

```
add_dependencies(first_program      ${first_program_EXPORTED_TARGETS}
${catkin_EXPORTED_TARGETS})
```

Aquesta línia afegeix totes les dependències de destinació cmake de l'executable. S'utilitza bàsicament per permetre que CMake compili correctament un paquet, assegurant-se que totes les dependències estan al seu lloc.

```
target_link_libraries(first_program
    ${catkin_LIBRARIES}
)
```

Aquesta línia especifica les llibreries que s'han d'utilitzar per enllaçar un objectiu determinat. En aquest cas, indica utilitzar les llibreries de catkin per enllaçar amb l'executable que heu creat.

9. Nodes de ROS

Heu iniciat un node al codi anterior, però... què és un node? Els nodes ROS són bàsicament programes fets en ROS. L'ordre ROS per veure quins nodes s'estan executant realment en un ordinador és:

```
T3: $ rostopic list
```

Si tot va bé, no haurieu de trobar el vostre node. Això és perquè el node acaba quan finalitza el programa C++. Canviem-ho. Recordeu en compilar un cop heu acabar de modificar-lo.

```
#include <ros/ros.h>

int main(int argc, char** argv) {

    ros::init(argc, argv, "Remembering_ROS");
    ros::NodeHandle nh;
    ros::Rate loop_rate(2); // We create a Rate object of 2Hz

    while (ros::ok()) // Endless loop until Ctrl + C
    {
        ROS_INFO("Trying to remember ROS....");
        ros::spinOnce();
        loop_rate.sleep(); // We sleep the needed time to maintain the Rate
fixed above
    }

    return 0;
}

// This program creates an endless loop that repeats itself 2 times per
second (2Hz) until somebody presses Ctrl + C
// in the Shell
```

Torneu a iniciar el programa amb l'ordre `roslaunch`. (Abans a totes les terminals escriviu `ctrl+c`).

```
T1: $ roscore
T2: $ roslaunch my_package my_package_launch_file.launch
```

```
T3: $ rosnode list
```

Ara sí que l'haurieu de trobar!

Per veure informació sobre el nostre node, podem utilitzar la següent comanda:

```
T3: $ rosnode info /Remembering_ROS
```

10. Parameter Server

Un servidor de paràmetres és un diccionari que ROS utilitza per emmagatzemar paràmetres. Aquests paràmetres poden ser utilitzats pels nodes en temps d'execució i normalment s'utilitzen per a dades estàtiques, com ara paràmetres de configuració.

Per obtenir una llista d'aquests paràmetres, podeu escriure:

```
T3: $ rosparam list
```

Per obtenir un valor d'un paràmetre concret, podeu escriure:

```
rosparam get <parameter_name>
```

I per establir un valor a un paràmetre, podeu escriure:

```
rosparam set <parameter_name>
```

11. ROSCORE

Per tal que tot això funcioni, hem de tenir un roscore en funcionament. El roscore és el procés principal que gestiona tot el sistema ROS. Sempre heu de tenir un roscore en funcionament per treballar amb ROS. L'ordre que llança un roscore és:

```
roscore
```

NOTA: a la plataforma que utilitzeu per a aquest curs, heu de llençar sempre **roscore** quan executeu un procés.

12. SIMULACIÓ EN ROS: GAZEBO

Aquesta simulació de Gazebo utilitza el paquet ROS Gazebo, per tant, s'ha d'instal·lar la versió adequada de Gazebo per a ROS1 Noetic abans d'executar aquesta instrucció.

12.1 Instal·lació del paquet de simulació

NOTA: Si feu servir el PC del laboratori no cal que instal·leu res

El paquet de **simulació TurtleBot3** requereix els paquets **turtlebot3** i **turtlebot3_msgs** com a requisit previ. Sense aquests paquets, la simulació no es pot iniciar.

Si us plau, seguiu les instruccions de configuració del PC si no heu instal·lat els paquets necessaris i els paquets dependents.

```
T1: $ cd ~/catkin_ws/src/  
$ git clone -b noetic-devel https://github.com/ROBOTIS-GIT/turtlebot3\_simulations.git  
$ cd ~/catkin_ws && catkin_make
```

2.2 Launch del món de simulació

Hi ha tres entorns de simulació per a TurtleBot3. Seleccioneu un d'aquests entorns per llançar Gazebo.

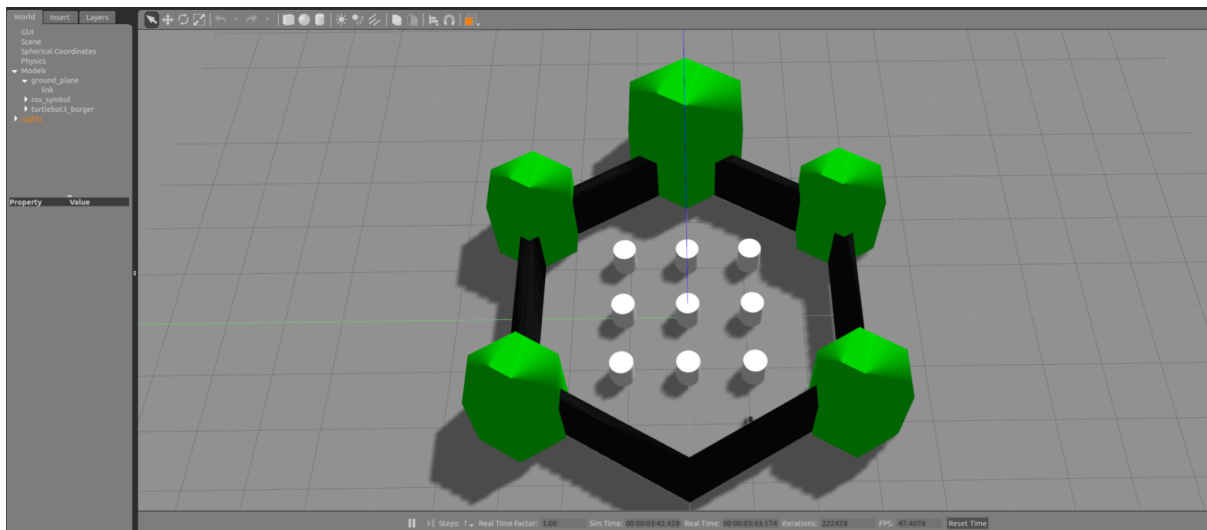
NOTA: Assegureu-vos d'acabar completament un món de simulació abans de llançar un món nou.

2.2.1 Empty World



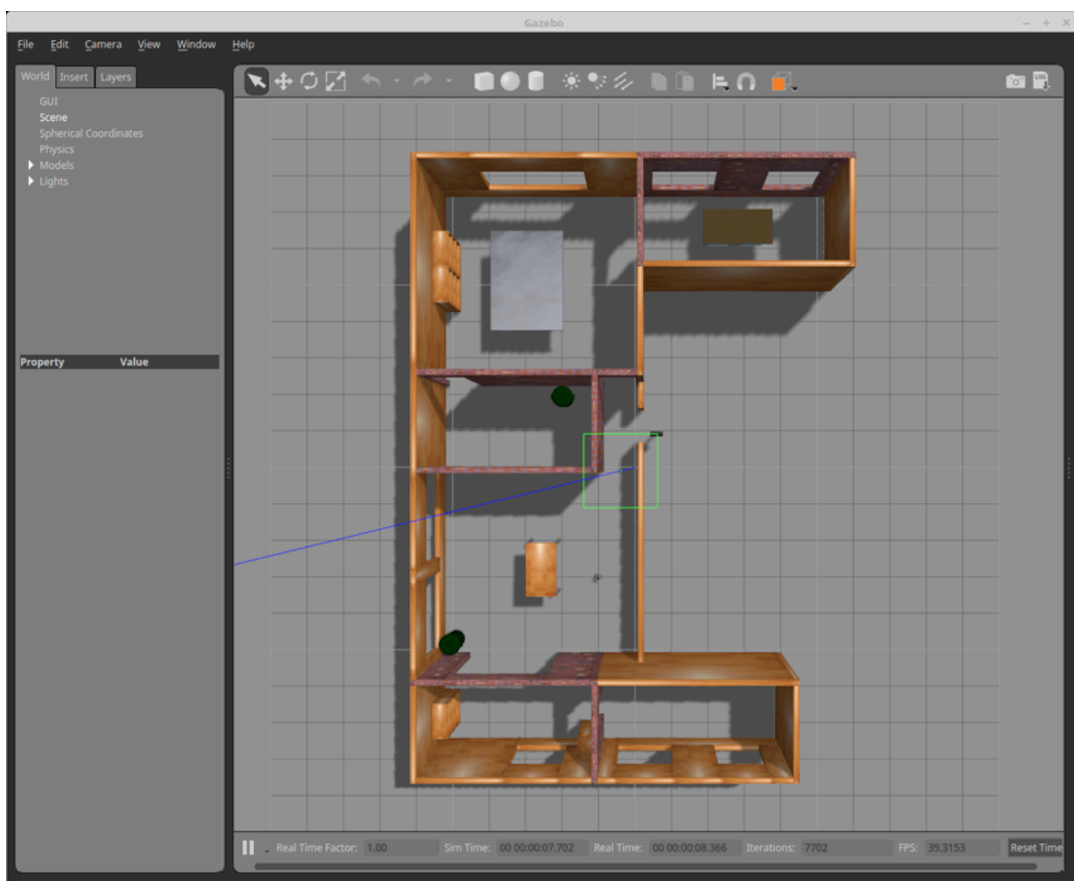
```
T1: $ roscore  
T2: $ roslaunch turtlebot3_gazebo turtlebot3_empty_world.launch
```

2.2.2 TurtleBot3 World



```
T1: $ roscore
T2: $ export TURTLEBOT3_MODEL=burger
    $ roslaunch turtlebot3_gazebo turtlebot3_world.launch
```

2.2.3 TurtleBot3 House



```
T1: $ roscore
T2: $ export TURTLEBOT3_MODEL=burger
$ roslaunch turtlebot3_gazebo turtlebot3_house.launch
```

NOTA: Si TurtleBot3 House s'inicia per primera vegada, la descàrrega del mapa pot trigar uns quants minuts, depenent de l'estat de la xarxa.

12.3 Operar el turtlebot3

Per tal de teleoperar el TurtleBot3 amb el teclat, inicieu el node de teleoperació amb l'ordre següent en una finestra de terminal nova.

```
T1: $ roscore
```

Trieu un dels tres mapes:

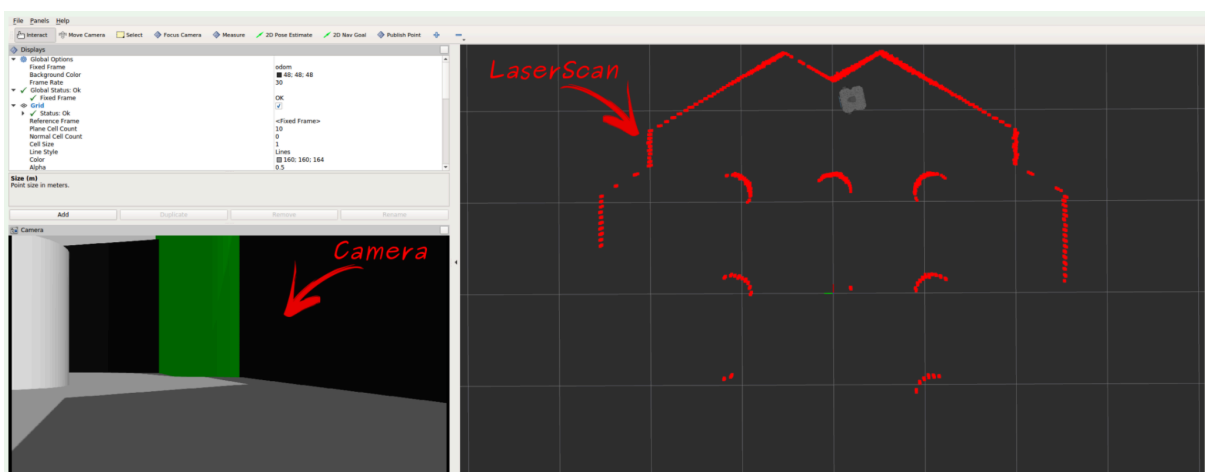
```
T2: $ roslaunch turtlebot3_gazebo turtlebot3_world.launch
T3: $ roslaunch turtlebot3_teleop turtlebot3_teleop_key.launch
```

Ara podeu navegar pel mapa controlant el robot amb teclat.

12.4 Visualització en RVIZ

RViz visualitza els **topics** publicats mentre s'executa la simulació. Podeu iniciar RViz en una nova finestra de terminal introduint l'ordre següent.

```
T4: $ roslaunch turtlebot3_gazebo turtlebot3_gazebo_rviz.launch
```



Aneu movent el robot en la terminal on heu llençat el teleop i mireu com va canviant el rviz, què creieu que està passant?

13. Simulació de SLAM

El SLAM (Simultaneous Localization and Mapping) és una tècnica per dibuixar un mapa estimant la ubicació actual en un espai arbitrari. El SLAM és una característica coneguda de TurtleBot dels seus predecessors. (Abans de continuar, recordeu parar tots els processos en totes les terminals, CTRL+C).

13.1 Launch demo món en simulació

Aquí es preparen els entorns Gazebo, però per crear un mapa amb SLAM, es recomana utilitzar **TurtleBot3 World** o **TurtleBot3 House**.

Utilitzeu una de les ordres següents per carregar l'entorn Gazebo. En aquesta instrucció, s'utilitzarà TurtleBot3 World.

```
T1: $ roscore
T2: $ roslaunch turtlebot3_gazebo turtlebot3_world.launch
```

13.2 Executar el node de SLAM

Obriu un nou terminal des del PC remot amb Ctrl + Alt + T i executeu el node SLAM. El mètode SLAM Gmapping s'utilitza per defecte.

```
T3: $ roslaunch turtlebot3_slam turtlebot3_slam.launch slam_methods:=gmapping
```

Si aquesta instrucció us dona error fer el següent:

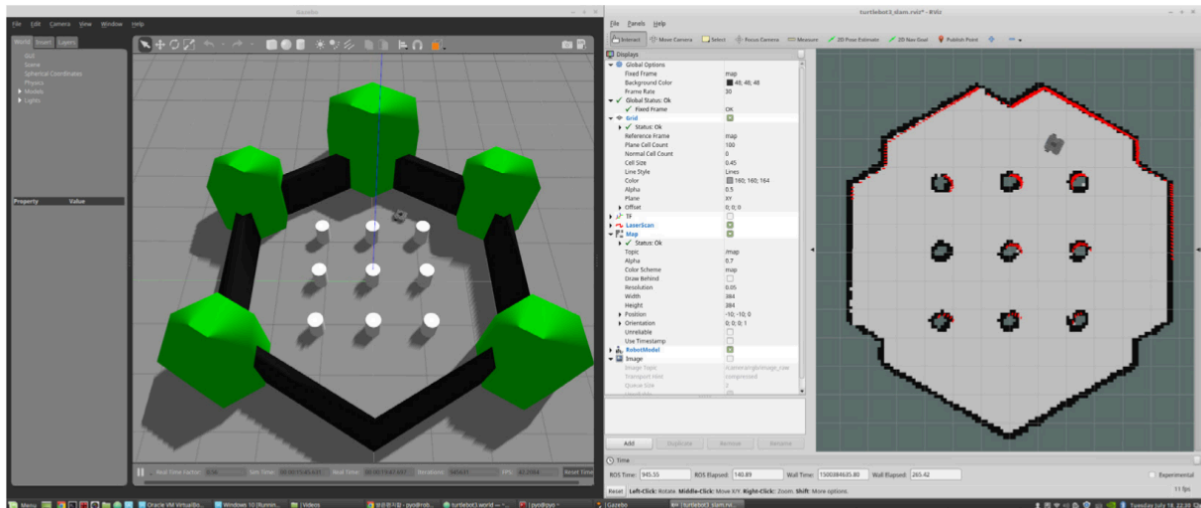
```
T3: $ export TURTLEBOT3_MODEL=burger
T3: $ roslaunch turtlebot3_slam turtlebot3_slam.launch slam_methods:=gmapping
```

13.3 Executar el node de teleoperació

Obriu un nou terminal des del PC remot amb Ctrl + Alt + T i executeu el node de teleoperació des del PC remot.

```
T4: $ roslaunch turtlebot3_teleop turtlebot3_teleop_key.launch
```

Aneu navegant pel mapa amb el teleop fins que estigui tot creat.



13.4 Guardem el mapa

Quan el mapa s'hagi creat correctament, obriu un nou terminal des del PC remot amb Ctrl + Alt + T i deseu el mapa.

T5: `$ rosrun map_server map_saver -f ~/NOM_DEL_MAPA`

14. Recopilatori

Heu vist diferents mètodes per a simular robots i no haver d'utilitzar el robot contínuament, són eines molt potents.

T1: `$ echo -e '\nexport TURTLEBOT3_MODEL=burger' >> ~/.bashrc && source ~/.bashrc`

T1: `$ roscore`

Executem el model de simulació del turtlebot 3 (existeixen diferents mapes)::

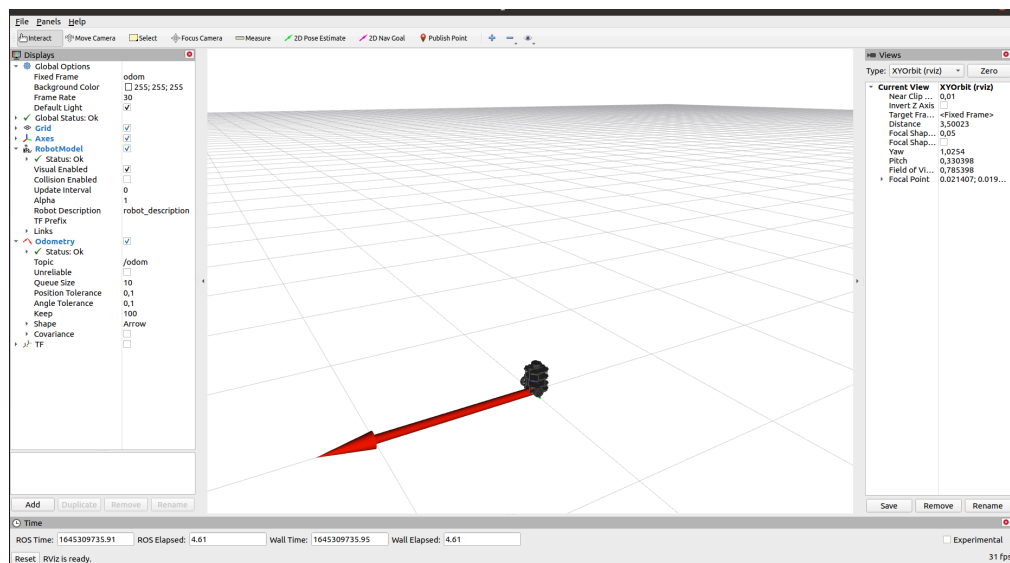
T2: `$ roslaunch turtlebot3_gazebo turtlebot3_world.launch`

Si voleu simular el vostre robot en un mapa creat anteriorment podeu fer el següent:

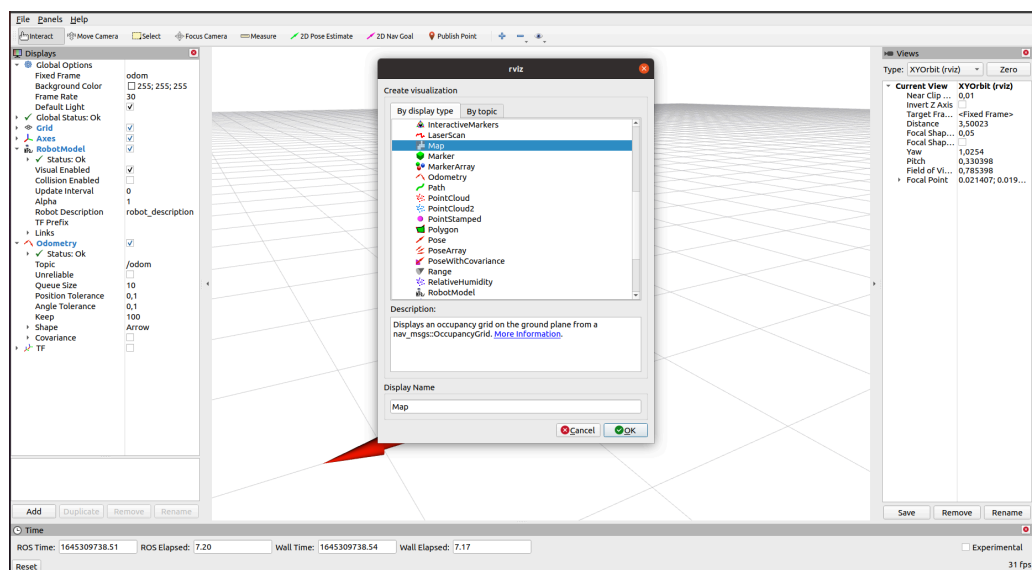
T3: `$ rosrun map_server map_server my_map.yaml`

Se us obrirà el rviz i aquí afegiu el topic **/map**

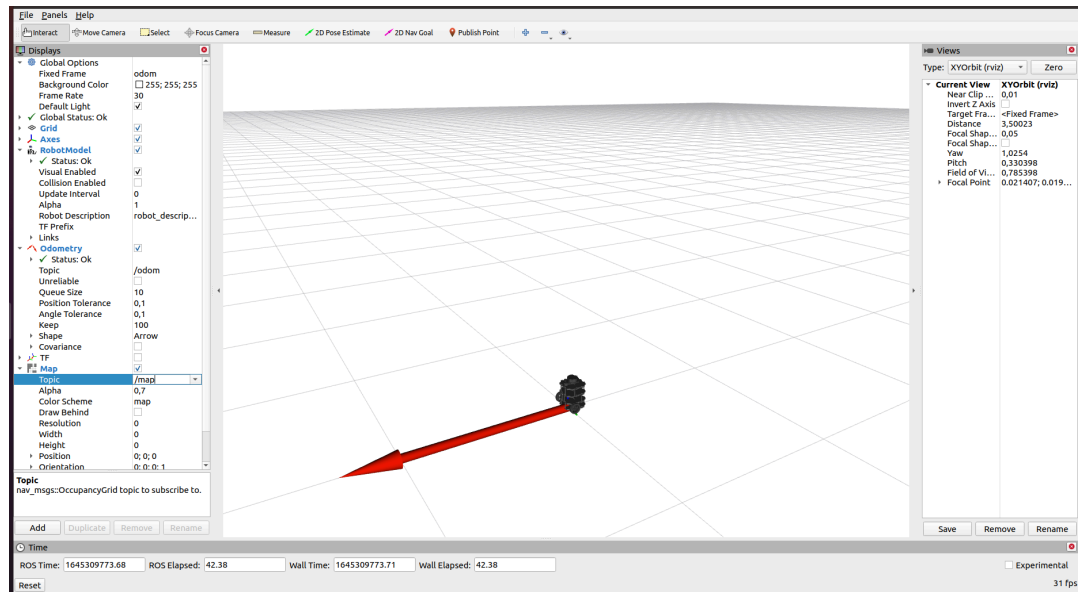
1. A la part inferior esquerra apreteu el botó **ADD**



2. Us apareixerà una finestra nova i escolliu: **map**



3. Escriviu **/map** a la casella de topic, us hauria de sortir un mapa.



Si voleu teleoperar el vostre robot:

T4: `$ roslaunch turtlebot3_teleop turtlebot3_teleop_key.launch`